# SISTEMAS INFORMÁTICOS - 1º DAW

# MATERIAL DE ESTUDIO COMPLEMENTARIO

## Unidad Didáctica 1: Hardware de un Sistema Informático

## Curso 2026-2027

**Elaborado por: José Fernández González**

**Versión 1.1 · octubre de 2026**

**CC BY-NC-ND 4.0**

**https://github.com/jstechg/material-docente/tree/main/1DAW_Sistemas_Informaticos**

# 1. Hardware de un sistema informático - Parte I

## 1.1. El sistema informático: elementos, finalidad y evolución

Antes de estudiar componentes concretos conviene disponer de una visión global de qué es un sistema informático, qué elementos lo forman y cómo han evolucionado las formas de organizar la computación.

Esta primera parte de la unidad se sitúa dentro del módulo Sistemas Informáticos y aborda el hardware de un sistema informático. El objetivo no es únicamente reconocer piezas de un ordenador, sino comprender cómo cooperan procesador, memoria, dispositivos de entrada/salida, buses, alimentación y almacenamiento para ejecutar programas y tratar información.

Para un estudiante de 1.º de DAW este conocimiento es útil aunque su perfil profesional esté orientado al desarrollo de software. El rendimiento de una aplicación, la gestión de memoria, la concurrencia, el acceso a disco, las comunicaciones y muchas decisiones de despliegue dependen de la arquitectura física subyacente. Por ello conviene estudiar el hardware como la base sobre la que se ejecutan el sistema operativo, los servidores, las herramientas de desarrollo y las aplicaciones web.

El recorrido de contenidos comienza definiendo qué es un sistema informático y repasando la clasificación clásica de ordenadores. Después se introduce la arquitectura hardware, el ciclo máquina y los principales componentes físicos: cajas, fuentes de alimentación, placas base, procesadores, memorias, periféricos y buses.

La parte final del índice anticipa la representación interna de la información, el montaje y la puesta en marcha. Es importante entender que estos contenidos no están aislados: la CPU ejecuta instrucciones almacenadas en memoria, se comunica mediante buses, recibe y entrega información a través del subsistema de entrada/salida y necesita una plataforma física compatible y correctamente alimentada. La representación binaria será el lenguaje común que conecta todos estos elementos.

La palabra informática suele explicarse como la unión de información y automática: el propósito de la informática es realizar de forma automática operaciones sobre información. Un sistema informático es el conjunto organizado de recursos que permite capturar datos, almacenarlos, procesarlos y obtener resultados. En su forma más simple, ese flujo puede describirse como entrada, almacenamiento, procesamiento y salida.

La entrada puede proceder de un teclado, un sensor, un fichero o una petición de red. El almacenamiento conserva temporal o permanentemente los datos. El procesamiento aplica instrucciones para transformarlos, y la salida presenta o comunica el resultado. En una aplicación web, por ejemplo, una petición HTTP puede considerarse entrada; una base de datos, almacenamiento; el código del servidor, procesamiento; y la respuesta HTML o JSON, salida.

Un sistema informático no debe confundirse con un único ordenador. Puede estar formado por varios equipos, redes, dispositivos, software y servicios coordinados. En entornos actuales es frecuente que una aplicación dependa de servidores, bases de datos, almacenamiento remoto y dispositivos cliente distribuidos.

El hardware engloba los componentes físicos: procesador, memoria, placa base, unidades de almacenamiento, pantalla, teclado, tarjetas de red, cableado, etc. El software comprende los programas y componentes lógicos que indican al hardware qué operaciones debe realizar: sistema operativo, controladores, aplicaciones, bibliotecas y herramientas de desarrollo.

Los datos constituyen el contenido sobre el que trabaja el sistema. Un mismo hardware y un mismo programa pueden producir resultados distintos según los datos recibidos. En un sistema web, el código de una aplicación puede ser idéntico para todos los usuarios, pero los datos de cada cuenta, los productos de una tienda o los mensajes almacenados determinan la información concreta que se procesa.

Por tanto, hardware, software y datos son conceptos diferentes pero inseparables en la práctica. El hardware aporta capacidad de ejecución, el software define procedimientos y los datos aportan el contenido que se transforma en información útil.

Además de hardware, software y datos, el elemento humano es fundamental: usuarios, administradores, desarrolladores, técnicos y responsables de seguridad diseñan, configuran, utilizan y mantienen el sistema. Sin personas que definan objetivos, reglas y procedimientos, el sistema carece de finalidad práctica.

También pueden considerarse parte del sistema los procedimientos de trabajo: copias de seguridad, políticas de acceso, procesos de despliegue, mantenimiento, recuperación ante fallos o actualización. En un entorno profesional, una infraestructura técnicamente correcta puede fallar por una mala operación humana o por procedimientos inadecuados.

Para DAW conviene relacionar esta idea con el ciclo de vida del software. El desarrollador no trabaja de forma aislada: su aplicación se ejecuta sobre una infraestructura, maneja datos reales y será utilizada y administrada por personas con necesidades concretas. Diseñar pensando en esos usuarios y en la operación del sistema forma parte de una visión completa de la informática.

Los primeros grandes sistemas de computación tendían a concentrar procesamiento, memoria y almacenamiento en equipos centrales de gran tamaño. Los usuarios accedían a esos recursos mediante terminales. Con la evolución del hardware, las redes y el software se generalizaron arquitecturas distribuidas, en las que diferentes máquinas o componentes cooperan a través de una red.

La palabra arquitectura describe la organización de un sistema y las relaciones entre sus componentes. Puede hablarse de arquitectura hardware para describir CPU, memoria, buses y E/S, y también de arquitectura software para describir cómo se divide una aplicación en módulos o servicios. Los ejemplos de arquitectura monolítica y microservicios pertenecen al ámbito del software, pero sirven para visualizar la misma idea general: definir componentes, responsabilidades y comunicaciones.

En la web actual es habitual distribuir funciones entre navegador, servidor de aplicaciones, base de datos, cachés, almacenamiento y servicios externos. Esta distribución ofrece escalabilidad y flexibilidad, pero también introduce latencia, fallos de red, sincronización y necesidades de seguridad.

La clasificación clásica distingue supercomputadores, mainframes, minicomputadores, estaciones de trabajo y ordenadores personales. La separación se basaba históricamente en potencia de cálculo, número de usuarios simultáneos, coste, fiabilidad y propósito. Hoy las fronteras son menos claras, pero la clasificación ayuda a comprender la evolución de la informática.

Los supercomputadores se diseñan para cálculo científico de muy alto rendimiento y suelen utilizar grandes cantidades de procesadores trabajando en paralelo. Los mainframes están optimizados para alta disponibilidad, procesamiento masivo de transacciones y gran número de operaciones de entrada/salida. Los minicomputadores ocuparon históricamente un nivel intermedio entre mainframes y microordenadores, y muchas de sus funciones han sido absorbidas por los servidores modernos.

Una workstation es un equipo personal de altas prestaciones destinado a tareas profesionales como ingeniería, CAD, simulación o creación de contenidos. El ordenador personal prioriza versatilidad y coste para un solo usuario. La potencia por sí sola ya no basta para clasificar un sistema: importan también disponibilidad, escalabilidad, especialización y forma de uso.

Además de la clasificación clásica, hoy conviene incluir portátiles, smartphones, tabletas, servidores, sistemas embebidos, dispositivos IoT y plataformas de computación en la nube. Un teléfono inteligente es un ordenador completo con procesador, memoria, almacenamiento, sistema operativo, sensores y comunicaciones. Un sistema embebido integra capacidad de cómputo dentro de otro producto, como un automóvil, un electrodoméstico o un dispositivo industrial.

Los servidores se diseñan para prestar servicios a otros equipos y suelen priorizar disponibilidad, capacidad de memoria, conectividad, almacenamiento y gestión remota. En la nube, esos recursos físicos se abstraen mediante virtualización y se ofrecen como máquinas virtuales, contenedores, bases de datos gestionadas u otros servicios.

Por tanto, una clasificación moderna puede hacerse por finalidad, número de usuarios, movilidad, consumo energético, nivel de integración, tolerancia a fallos o capacidad de procesamiento. Esta perspectiva invita a superar una clasificación puramente histórica.

## 1.2. Arquitectura hardware y modelo de von Neumann

La arquitectura hardware permite entender el ordenador como un conjunto de bloques funcionales que cooperan mediante mecanismos de comunicación bien definidos.

El esquema representa una arquitectura de computador basada en grandes bloques funcionales: CPU, memoria principal y sistemas de entrada/salida. La CPU contiene registros, unidad de control y unidad aritmético-lógica (ALU). La memoria conserva instrucciones y datos, mientras que la entrada/salida permite comunicarse con dispositivos externos.

Estos bloques intercambian información mediante caminos de comunicación que, de forma conceptual, se agrupan en buses. La utilidad del modelo es separar funciones: la memoria almacena, la CPU interpreta y ejecuta instrucciones y el subsistema de E/S conecta el computador con el exterior.

Los procesadores actuales son mucho más complejos que el dibujo: incluyen varios núcleos, múltiples niveles de caché, predicción de saltos, ejecución fuera de orden y controladores integrados. Sin embargo, el modelo simplificado sigue siendo muy útil para comprender cómo se ejecuta un programa.

La arquitectura de von Neumann se asocia al concepto de programa almacenado: las instrucciones de un programa se representan como información y se almacenan en memoria junto con los datos. La CPU obtiene sucesivamente esas instrucciones, las decodifica y las ejecuta. Esta idea permitió construir computadores de propósito general capaces de ejecutar programas diferentes sin modificar físicamente su cableado.

El modelo clásico utiliza una memoria común y un camino compartido para instrucciones y datos. Esto conduce al llamado cuello de botella de von Neumann: el rendimiento puede quedar limitado por la velocidad con la que la CPU intercambia información con la memoria. Los sistemas actuales mitigan este problema mediante cachés, buses rápidos, ejecución paralela y otras técnicas.

No todos los computadores siguen de forma pura este modelo. Existen arquitecturas Harvard, con memorias o caminos separados para instrucciones y datos, y muchos procesadores modernos utilizan soluciones híbridas, por ejemplo cachés separadas de instrucciones y datos en los niveles más próximos al núcleo.

La CPU ejecuta instrucciones y coordina la actividad del sistema. La memoria principal mantiene los datos y las instrucciones que deben estar accesibles con baja latencia durante la ejecución. El sistema de entrada/salida conecta el procesador y la memoria con periféricos como teclado, pantalla, almacenamiento, red o sensores.

Los buses son mecanismos de interconexión. Conceptualmente se distingue el bus de datos, por el que circulan valores e instrucciones; el bus de direcciones, que identifica posiciones de memoria o dispositivos; y el bus de control, que transporta señales de coordinación como lectura, escritura, interrupciones o sincronización.

En equipos modernos estas funciones no siempre corresponden a un único bus físico compartido. Tecnologías como PCI Express utilizan enlaces serie punto a punto, y algunos controladores están integrados en la propia CPU. Aun así, la separación conceptual sigue siendo válida para comprender qué tipo de información debe circular entre los componentes.

## 1.3. Procesador y ciclo de ejecución de instrucciones

El procesador es el elemento que interpreta y ejecuta las instrucciones de los programas. Para comprender su funcionamiento es necesario relacionar sus unidades internas con el ciclo básico de búsqueda, decodificación, ejecución y almacenamiento de resultados.

La CPU es el componente encargado de interpretar y ejecutar las instrucciones de los programas. Su función no consiste únicamente en realizar cálculos: también toma decisiones, mueve datos entre registros y memoria, controla saltos del programa y coordina operaciones con otros subsistemas.

Cuando se dice que la CPU procesa instrucciones desde la memoria, se resume un ciclo continuo. La CPU obtiene una instrucción, determina qué operación representa, localiza los operandos necesarios, ejecuta la operación y guarda el resultado. Este proceso se repite a enorme velocidad.

La frecuencia de reloj es solo uno de los factores del rendimiento. Dos procesadores a la misma frecuencia pueden ejecutar cantidades de trabajo muy distintas por ciclo debido a diferencias de arquitectura, número de núcleos, tamaño de caché, ancho de ejecución, predicción de saltos y conjunto de instrucciones.

La unidad de control interpreta las instrucciones y genera las señales necesarias para coordinar los restantes bloques de la CPU. La ALU realiza operaciones aritméticas y lógicas, como sumas, restas, comparaciones, desplazamientos y operaciones booleanas. Los registros son memorias internas extremadamente rápidas usadas para guardar temporalmente datos, direcciones e información de control.

El contador de programa (PC) contiene la dirección de la siguiente instrucción que debe ejecutarse. El registro de instrucción (IR) conserva la instrucción actual mientras se decodifica o ejecuta. En modelos didácticos se utiliza el MAR (Memory Address Register) para representar la dirección de memoria a la que se accede y el MDR o registro de datos para representar el dato transferido desde o hacia memoria.

Los acumuladores son registros asociados históricamente a resultados aritméticos. En arquitecturas actuales suele existir un conjunto más amplio de registros de propósito general, además de registros de estado, puntero de pila, registros vectoriales y otros especializados. Los nombres exactos dependen de la arquitectura, pero la idea esencial es que los registros son el almacenamiento de trabajo más rápido del procesador.

El ciclo máquina describe de forma simplificada cómo la CPU ejecuta un programa. En la fase de búsqueda (fetch), la dirección del contador de programa se utiliza para obtener de memoria la siguiente instrucción. Después se actualiza el contador para apuntar a la instrucción siguiente, salvo que posteriormente un salto cambie el flujo.

En la decodificación, la unidad de control interpreta el código de operación y determina qué operandos y recursos se necesitan. En la ejecución, la CPU realiza la operación: cálculo aritmético, acceso a memoria, comparación, salto o comunicación con otro subsistema. Finalmente, si procede, se almacena el resultado en un registro o en memoria.

Los procesadores reales superponen muchas instrucciones mediante pipelines y pueden ejecutar varias en paralelo o fuera de orden. Aun así, fetch-decode-execute es el modelo conceptual adecuado para comprender de dónde salen las instrucciones y cómo se transforma un programa almacenado en acciones físicas.

## 1.4. Memoria principal y jerarquía de memoria

El rendimiento de un sistema depende en gran medida de cómo se organizan y combinan distintos tipos de memoria con capacidades, velocidades y costes muy diferentes.

Durante la ejecución, las instrucciones y los datos de uso inmediato deben estar disponibles para el procesador. La memoria RAM es el componente principal para este propósito: es de acceso aleatorio, ofrece tiempos de acceso reducidos frente al almacenamiento secundario y, en los equipos habituales, es volátil, por lo que pierde su contenido cuando deja de recibir alimentación.

Dentro de la jerarquía de memoria conviene distinguir RAM, ROM, caché y registros. La ROM o memoria no volátil se utiliza para conservar firmware u otra información persistente. La caché es una memoria mucho más rápida y pequeña situada cerca o dentro de la CPU. Los registros están todavía más próximos a las unidades de ejecución. Todos forman parte de la jerarquía de memoria, aunque en sentido estricto no se consideran todos "memoria principal".

La razón de esta diversidad es económica y tecnológica: no existe una memoria que sea al mismo tiempo enorme, permanente, extremadamente rápida y barata. Por ello los sistemas combinan varios niveles con características diferentes.

La jerarquía de memoria ordena los distintos niveles según proximidad al procesador, velocidad, capacidad y coste por bit. En la parte superior se sitúan los registros y las cachés: son muy rápidos pero de capacidad reducida. A continuación aparece la RAM. Más abajo se encuentran SSD, discos duros y otros medios de almacenamiento, mucho más capaces pero también más lentos.

La jerarquía funciona gracias al principio de localidad. Los programas tienden a reutilizar datos e instrucciones recientemente usados (localidad temporal) y a acceder a posiciones próximas (localidad espacial). Las cachés aprovechan estos patrones guardando cerca del procesador la información que probablemente se necesitará de nuevo.

Cuando un dato está en un nivel rápido se obtiene con poca latencia; si no está, debe recuperarse de un nivel inferior y más lento. Por eso el rendimiento real de un sistema depende tanto del procesador como de la eficiencia con la que se mueve la información a través de la jerarquía.

Una jerarquía de memoria típica sitúa los registros y las cachés en los niveles más próximos al procesador, seguidos por la memoria principal y, a mayor distancia, por el almacenamiento persistente. Los niveles L1, L2 y L3 de caché suelen diferir en tamaño, latencia y grado de compartición entre núcleos: L1 suele ser la más pequeña y rápida, mientras que L3 acostumbra a ser mayor y puede compartirse entre varios núcleos.

La memoria virtual no es exactamente un nuevo tipo de memoria física situado entre RAM y disco. Es una técnica gestionada por hardware y sistema operativo que proporciona a cada proceso un espacio de direcciones virtual. Cuando falta RAM, algunas páginas pueden almacenarse temporalmente en un dispositivo de almacenamiento, pero este mecanismo es mucho más lento que la RAM y un uso intensivo provoca una degradación importante del rendimiento.

Los SSD reducen mucho la latencia frente a los discos duros, pero siguen estando varios órdenes de magnitud por debajo de los niveles internos del procesador. Por eso aumentar RAM puede mejorar enormemente un equipo que estaba utilizando intercambio de forma continua.

## 1.5. Cajas, factores de forma y fuente de alimentación

Los componentes electrónicos necesitan un soporte físico compatible y una alimentación estable. La caja, el factor de forma y la fuente de alimentación condicionan tanto el montaje inicial como las posibilidades de ampliación del equipo.

La caja, torre o chasis es la estructura física que contiene y protege los componentes del ordenador. No debe llamarse CPU al conjunto de la caja: la CPU es específicamente el procesador. Esta confusión es frecuente en el lenguaje cotidiano, pero conviene utilizar la terminología técnica correcta.

La caja cumple funciones mecánicas, térmicas y de compatibilidad. Proporciona puntos de montaje para placa base, fuente, almacenamiento, ventiladores y tarjetas de expansión; guía el flujo de aire; reduce riesgos de contacto accidental; y ofrece conectores y botones accesibles desde el exterior.

Los materiales habituales son acero, aluminio y plásticos. En equipos personalizados pueden utilizarse paneles de vidrio, iluminación o modificaciones estéticas, pero cualquier diseño debe mantener ventilación, rigidez, seguridad y compatibilidad con los componentes.

Las cajas se fabrican con distintos tamaños y orientaciones. Las torres y semitorres son comunes en ordenadores de escritorio; los equipos compactos reducen volumen y capacidad de expansión; los chasis de servidor suelen estar diseñados para instalarse en racks normalizados y facilitar mantenimiento, refrigeración y sustitución de componentes.

Un barebone es un sistema parcialmente montado que incluye algunos elementos básicos -por ejemplo caja, placa base y fuente- y que debe completarse con memoria, almacenamiento, procesador u otros componentes según el producto. Es importante consultar qué piezas incorpora realmente cada modelo, porque el término no garantiza una configuración idéntica.

La elección de la caja debe considerar el factor de forma de la placa, longitud y grosor de tarjetas gráficas, altura del disipador, radiadores, número de unidades, potencia térmica y posibilidades de expansión. Una caja demasiado pequeña puede ser compatible sobre el papel y, sin embargo, dificultar el montaje o la refrigeración.

En una caja de sobremesa tradicional, la zona posterior reúne normalmente la fuente de alimentación, los conectores externos de la placa base, las ranuras de expansión y las salidas de ventilación. En el interior se distribuyen la placa base, las unidades de almacenamiento, los ventiladores y el cableado de alimentación y del panel frontal. La disposición exacta depende del diseño de cada chasis.

En equipos actuales han cambiado algunos detalles. Las antiguas bahías externas de 5,25 pulgadas y muchas de 3,5 pulgadas son menos frecuentes, mientras que han ganado importancia las posiciones para SSD, la gestión de cables, los ventiladores de gran diámetro y los radiadores de refrigeración líquida. Los conectores del panel frontal suelen incluir USB, audio, botón de encendido y LED de estado.

Al montar un equipo hay que mantener despejadas las entradas y salidas de aire y evitar cables que puedan rozar ventiladores. También deben utilizarse los separadores correctos bajo la placa base para impedir cortocircuitos contra el chasis.

El factor de forma define dimensiones y reglas de compatibilidad entre componentes. En una placa base establece tamaño, posición de tornillos, zona de conectores y distribución general. En la fuente de alimentación define dimensiones y puntos de fijación. La caja debe admitir los factores de forma de los elementos que se quieran instalar.

ATX es uno de los estándares más extendidos en equipos de escritorio. Micro-ATX reduce el tamaño y normalmente el número de ranuras de expansión. Mini-ITX es todavía más compacto y se utiliza en equipos de pequeño formato. Una caja ATX suele admitir placas más pequeñas, pero siempre debe comprobarse la especificación concreta.

El factor de forma no determina por sí solo el rendimiento. Una placa Mini-ITX puede incorporar un procesador muy potente, pero dispone de menos espacio para ranuras, conectores y refrigeración. Por ello la elección depende del equilibrio entre tamaño, capacidad de expansión, coste y necesidades térmicas.

Entre los factores de forma más habituales en equipos de escritorio se encuentran ATX, Micro-ATX y Mini-ITX. Como referencia, una placa ATX estándar mide aproximadamente 305 x 244 mm; Micro-ATX admite hasta unos 244 x 244 mm; y Mini-ITX mide 170 x 170 mm. Estas dimensiones ayudan a anticipar compatibilidades, pero no sustituyen la consulta de las especificaciones concretas de la caja y de la placa base.

La posición de los conectores y los puntos de montaje está normalizada para facilitar la interoperabilidad, pero las dimensiones de componentes adicionales pueden provocar incompatibilidades. Por ejemplo, una tarjeta gráfica puede ser demasiado larga o gruesa para una caja, o un disipador puede superar la altura disponible.

A lo largo de la historia han existido otros factores de forma, algunos hoy poco frecuentes. En un montaje real no debe suponerse que dos componentes son compatibles solo por pertenecer a una familia similar: siempre se comprueban las dimensiones, los puntos de fijación, los conectores y las restricciones indicadas por el fabricante.

La fuente de alimentación convierte la corriente alterna de la red eléctrica en tensiones continuas reguladas que pueden utilizar los componentes del ordenador. En España y gran parte de Europa la tensión nominal de red actual es 230 V, aunque en materiales antiguos aparezca a veces 220 V. Las líneas más relevantes de una fuente ATX moderna son +12 V, +5 V y +3,3 V.

La fuente debe proporcionar potencia suficiente y mantener las tensiones dentro de los márgenes permitidos incluso cuando cambia la carga. Una fuente insuficiente o de mala calidad puede provocar reinicios, inestabilidad, pérdida de datos o daños. La potencia nominal en vatios no es el único criterio: importan eficiencia, calidad de regulación, capacidad de la línea de 12 V y protecciones eléctricas.

La capacidad de expansión depende de la fuente porque añadir una GPU, discos, ventiladores o tarjetas incrementa el consumo y puede requerir conectores específicos. Conviene elegir margen razonable sin sobredimensionar de forma extrema y valorar certificaciones de eficiencia y protecciones frente a sobretensión, sobrecorriente, cortocircuito o temperatura.

Las especificaciones ATX antiguas contemplaron una línea de -5 V, pero esta dejó de formar parte de las fuentes ATX modernas habituales. En equipos actuales, para una introducción didáctica, las líneas de continua más relevantes son +12 V, +5 V y +3,3 V.

En cableado ATX tradicional, amarillo suele corresponder a +12 V, rojo a +5 V y negro a COM o referencia de 0 V. Es importante no identificar el conductor negro con la toma de tierra de protección: COM es el retorno eléctrico de las salidas de corriente continua, mientras que la tierra de protección está conectada al chasis y a la instalación eléctrica de red.

Las fuentes ATX se controlan mediante señales específicas. La señal PS_ON#, tradicionalmente asociada al conductor verde del conector principal, permite ordenar el encendido cuando se lleva a nivel bajo respecto a COM. Por eso técnicamente es posible arrancar una fuente sin placa base puenteando la señal adecuada, pero esta operación solo debe realizarse con conocimiento del pinout, la fuente desconectada durante la preparación y evitando cualquier contacto con la parte interna.

Nunca debe abrirse una fuente de alimentación para prácticas ordinarias. En su interior existen condensadores y circuitos conectados a tensión de red que pueden resultar peligrosos incluso después de desenchufarla. Además, en fuentes modulares no debe suponerse que los conectores del lado de la fuente tienen el mismo pinout entre fabricantes o modelos.

## 1.6. Memoria RAM: prestaciones, fiabilidad y generaciones

Una vez situada la RAM dentro de la jerarquía de memoria, interesa estudiar las características que permiten comparar módulos reales: volatilidad, ancho de banda, latencia, corrección de errores, tecnología interna y compatibilidad física.

Para comparar módulos de memoria no basta con indicar su capacidad. La volatilidad describe si conservan datos sin alimentación; la velocidad expresa la tasa a la que pueden transferirse datos; ECC indica si existe capacidad de detección y corrección de determinados errores; y la latencia CAS refleja una parte del tiempo necesario para responder a una lectura.

Estas características están relacionadas. Una memoria con mayor tasa de transferencia puede mover más datos por segundo, pero el tiempo de respuesta también depende de latencias medidas en ciclos. Por eso dos módulos con distinta generación o frecuencia no pueden compararse únicamente por el número CL.

Además hay que comprobar compatibilidad: generación DDR, tipo de módulo, capacidad soportada por placa y CPU, número de canales, perfiles de velocidad y soporte de ECC. El rendimiento final depende del conjunto del subsistema de memoria.

Una memoria es volátil cuando necesita alimentación para conservar la información. La RAM principal utilizada por los programas es volátil: al apagar el equipo se pierde su contenido. Esta característica permite tecnologías

rápidas y adecuadas para el trabajo temporal, pero obliga a guardar en almacenamiento no volátil cualquier información que deba persistir.

SSD, discos duros y memorias flash son no volátiles y conservan los datos sin energía. También existe firmware almacenado en memorias no volátiles de la placa base y de otros dispositivos. El sistema operativo copia desde almacenamiento a RAM los programas y datos que deben utilizarse con rapidez.

Comprender la diferencia es esencial para interpretar acciones cotidianas: guardar un documento significa escribirlo en almacenamiento persistente; tenerlo abierto implica que parte de sus datos reside además en memoria de trabajo.

El ancho de banda indica cuántos datos pueden transferirse por unidad de tiempo. Un canal de memoria convencional presenta un ancho lógico de 64 bits de datos, es decir, 8 bytes por transferencia. Para estimar el ancho de banda teórico se multiplican esos 8 bytes por la tasa efectiva de transferencias.

En memorias DDR es más preciso hablar de MT/s (millones de transferencias por segundo) que de MHz efectivos. Por ejemplo, un canal que opere a 1600 MT/s tiene un ancho de banda teórico de 8 bytes x 1600 millones de transferencias por segundo = 12,8 GB/s. La frecuencia física del reloj no es necesariamente 1600 MHz, porque DDR realiza transferencias en ambos flancos del reloj.

Si el sistema utiliza varios canales en paralelo, el ancho de banda agregado puede aumentar. Sin embargo, el rendimiento real es menor que el máximo teórico por sobrecarga, patrones de acceso, latencia y limitaciones del controlador.

Conviene recordar que 64 bits equivalen a 8 bytes, porque 1 byte = 8 bits.

Un módulo identificado como PC3-12800 utiliza una nomenclatura habitual para DDR3-1600. El cálculo didáctico es correcto si se interpreta 1600 como tasa efectiva de 1600 MT/s: un canal de 64 bits equivale a 8 bytes, y 8 x 1600 millones da 12.800 MB/s, aproximadamente 12,8 GB/s en notación decimal.

La denominación comercial PC3-12800 procede precisamente de ese ancho de banda teórico aproximado. Conviene distinguirlo de la capacidad del módulo: 8 GB de capacidad indica cuántos datos puede almacenar, mientras que 12,8 GB/s indica cuántos datos podría transferir por segundo en condiciones ideales.

También hay que diferenciar MHz y MT/s. En DDR3-1600 el reloj base de la interfaz es aproximadamente 800 MHz, pero se realizan dos transferencias por ciclo, de ahí los 1600 MT/s. Esta precisión evita errores al comparar generaciones modernas de memoria.

ECC significa Error Correcting Code. La memoria ECC incorpora información redundante que permite detectar y, según el esquema utilizado, corregir determinados errores en los bits almacenados. En una implementación SECDED típica se puede corregir un error de un solo bit y detectar errores de dos bits, aunque existen esquemas de protección más avanzados.

Los errores de memoria pueden deberse a fallos eléctricos, radiación, envejecimiento u otras causas. En un ordenador doméstico son poco frecuentes, pero en servidores que funcionan continuamente y manejan grandes cantidades de memoria el riesgo acumulado es mayor; por eso ECC es habitual en sistemas donde la integridad y disponibilidad son prioritarias.

El uso de ECC requiere soporte del controlador de memoria y de la plataforma, no solo comprar un módulo diferente. Además existen módulos registrados o buffered para servidores que no deben confundirse con ECC: son características distintas, aunque a menudo aparezcan juntas.

La gestión de los errores ECC depende de la plataforma. Un error corregible puede resolverse de forma transparente y quedar registrado, mientras que otros errores pueden notificarse al sistema operativo o provocar una respuesta más severa. Por ello no debe asociarse ECC a una única reacción fija.

La latencia CAS (CL) expresa el número de ciclos de reloj que transcurren entre una solicitud de lectura de una columna de memoria y la disponibilidad de los datos, dentro de un conjunto concreto de condiciones. Un CL

menor significa menos ciclos, pero no puede compararse aisladamente entre memorias que trabajan a velocidades diferentes.

Para comparar mejor puede estimarse la latencia temporal. Una memoria con más ciclos CAS puede tener un ciclo mucho más corto y, por tanto, un tiempo real similar o incluso menor. Al mismo tiempo, una tasa de transferencia más alta mejora el movimiento de bloques grandes de datos. Por eso latencia y ancho de banda son métricas complementarias.

Los valores concretos de DDR, DDR2, DDR3 y DDR4 usados como ejemplos históricos no deben tratarse como rangos universales. Existen numerosos perfiles dentro de cada generación y DDR5 utiliza números CL mayores, compensados en parte por tasas de transferencia mucho más altas.

La RAM puede construirse con distintas tecnologías. La SRAM (Static RAM) mantiene cada bit mediante circuitos biestables mientras recibe alimentación. Es muy rápida y no necesita ciclos de refresco, pero ocupa más área de silicio y resulta cara por bit. Por ello se utiliza principalmente en memorias caché y estructuras internas del procesador.

La DRAM (Dynamic RAM) almacena la información como carga eléctrica en condensadores que se descargan gradualmente, de modo que es necesario refrescarla periódicamente. Es más densa y económica, razón por la que se utiliza como memoria principal de gran capacidad.

SDRAM es DRAM sincronizada con un reloj, y DDR SDRAM transfiere datos en ambos flancos de ese reloj. Las generaciones DDR, DDR2, DDR3, DDR4 y DDR5 evolucionan en velocidad, señalización, voltaje y arquitectura, y no son intercambiables físicamente.

Los módulos DIMM de generaciones DDR y DDR2 ilustran cómo evoluciona el encapsulado físico de la memoria. Un DIMM es una placa de circuito que agrupa chips de memoria y presenta contactos eléctricos para conectarse a la placa base. Las generaciones se diferencian por número y posición de contactos, muescas mecánicas, tensiones y protocolo eléctrico.

La muesca evita, en condiciones normales, insertar un módulo de una generación en un zócalo incompatible. DDR de escritorio utilizó habitualmente 184 contactos y DDR2 240. Aunque DDR2 y DDR3 compartan 240 contactos, la posición de la muesca y sus características eléctricas son distintas, por lo que no deben mezclarse.

Además de DIMM para equipos de escritorio existen formatos más compactos, como SO-DIMM para portátiles y sistemas pequeños. La compatibilidad depende del tipo exacto de módulo aceptado por la placa y el procesador.

DDR3 de escritorio utiliza normalmente DIMM de 240 contactos. DDR4 y DDR5 utilizan DIMM de 288 contactos, pero no son compatibles entre sí: la muesca, la señalización, las tensiones y la arquitectura interna son diferentes. Forzar un módulo puede dañar el zócalo o la memoria.

DDR5 aumenta considerablemente las tasas de transferencia y reorganiza el módulo en dos subcanales de 32 bits de datos, lo que mejora la eficiencia de acceso. En muchos módulos DDR5 la gestión de alimentación incorpora un PMIC en el propio módulo. También existe corrección interna dentro de los chips (on-die ECC), que no equivale a disponer de ECC completo de extremo a extremo como el utilizado por plataformas de servidor.

Al actualizar memoria hay que comprobar generación, formato DIMM o SO-DIMM, capacidad máxima, número de módulos, velocidades admitidas y perfiles de configuración. La placa base y el controlador de memoria de la CPU determinan qué combinaciones son válidas.

## 1.7. Periféricos y subsistema de entrada/salida

La entrada/salida conecta el procesador y la memoria con el exterior. Su estudio permite comprender por qué un periférico necesita componentes hardware, controladores y software específico para integrarse correctamente en el sistema operativo.

Los periféricos permiten que el sistema informático intercambie información con el exterior. Los de entrada introducen información: teclado, ratón, micrófono, cámara o lector de códigos. Los de salida presentan resultados: monitor, impresora o altavoces. Muchos dispositivos son bidireccionales y realizan entrada y salida, como una tarjeta de red, una pantalla táctil o una unidad de almacenamiento.

La clasificación depende de la dirección de la información respecto al sistema. Por ejemplo, una unidad SSD recibe datos cuando se escribe y entrega datos cuando se lee. Una interfaz de red transmite y recibe paquetes. Esta perspectiva evita clasificar dispositivos únicamente por su aspecto físico.

El sistema operativo proporciona abstracciones para que los programas utilicen los periféricos sin conocer todos sus detalles electrónicos. Esa separación se implementa mediante controladores de dispositivo, subsistemas de E/S y controladores hardware, conceptos que se desarrollan más adelante.

Un periférico es un dispositivo que permite entrada, salida, almacenamiento o comunicación con el sistema. Puede contener una parte mecánica o física -motores, sensores, cabezales, botones, paneles- y una parte electrónica que controla su funcionamiento y lo conecta con el computador.

El controlador hardware del dispositivo traduce señales del bus o interfaz a operaciones propias del periférico. Por ejemplo, un controlador USB o una controladora de almacenamiento gestiona detalles temporales y eléctricos que no debe resolver directamente cada aplicación.

La frontera entre periférico y componente interno es funcional, no necesariamente externa. Un SSD NVMe instalado en la placa base sigue formando parte del subsistema de E/S y almacenamiento aunque esté dentro de la caja.

Puede entenderse el subsistema de entrada/salida como una cadena de capas. El sistema operativo ofrece interfaces comunes a los programas; los drivers traducen esas operaciones a comandos específicos; y los controladores hardware gestionan la comunicación eléctrica y temporal con el periférico. Esta separación permite que una aplicación utilice dispositivos muy diferentes mediante abstracciones relativamente uniformes.

El driver conoce registros, comandos, colas, interrupciones y características del dispositivo, mientras que el sistema operativo presenta abstracciones más uniformes, como archivos, sockets o dispositivos de bloque. Esto permite que una aplicación pueda leer un fichero sin conocer si está almacenado en un SSD SATA, un NVMe o un recurso remoto.

No debe confundirse driver con controlador hardware. El driver es software; el controlador es lógica electrónica. Ambos cooperan para convertir una operación solicitada por el sistema en una acción real del dispositivo.

En la E/S controlada por programa, la CPU consulta repetidamente el estado del dispositivo y realiza directamente las transferencias. Es un mecanismo sencillo, pero puede desperdiciar mucho tiempo de procesador esperando. A esta espera activa se la suele llamar polling.

En la E/S por interrupciones, la CPU inicia la operación y continúa con otro trabajo. Cuando el dispositivo necesita atención o termina, genera una interrupción. El procesador pausa temporalmente el flujo normal, ejecuta una rutina de servicio y después continúa. Esto reduce la espera activa.

DMA (Direct Memory Access) permite que un controlador transfiera bloques de datos entre dispositivo y memoria con intervención mucho menor de la CPU. El procesador configura la operación y recibe una notificación al finalizar. DMA es especialmente útil en red, almacenamiento, audio o vídeo, donde se mueven grandes cantidades de datos.

La diferencia fundamental es cuánta intervención necesita la CPU durante la transferencia: mucha en E/S programada, puntual en interrupciones y reducida durante el movimiento de bloques mediante DMA.

## 1.8. Buses e interconexiones: datos, direcciones, control y transmisión

Los componentes no trabajan de forma aislada: necesitan intercambiar datos, direcciones y señales de control. Las interconexiones modernas explican también la transición histórica desde muchos buses paralelos hacia enlaces serie de alta velocidad.

El bus de datos transporta la información que se está transfiriendo: operandos, resultados e instrucciones. Su anchura indica cuántos bits pueden desplazarse en paralelo en una transferencia conceptual. El bus de direcciones especifica qué posición de memoria o qué recurso se desea acceder. Un mayor número de bits de dirección permite identificar un espacio de direcciones mayor.

El bus de control transporta señales que indican qué debe hacerse y cuándo: lectura, escritura, confirmaciones, peticiones de interrupción, arbitraje o sincronización. Sin estas señales, datos y direcciones no serían suficientes para coordinar los intercambios.

En hardware moderno el término bus se utiliza de forma más amplia. PCI Express, SATA o USB son interconexiones serie y no se parecen físicamente al bus paralelo compartido de los modelos clásicos, pero siguen resolviendo el mismo problema básico: transportar información y señales de control entre componentes.

Conceptualmente se distinguen buses de datos, direcciones y control según la función de la información transportada. También puede clasificarse una interconexión por su forma de transmisión. En paralelo se envían varios bits simultáneamente por varias líneas; en serie se envían secuencialmente por una o unas pocas parejas de conductores.

Las arquitecturas antiguas utilizaron numerosos buses paralelos porque era una forma directa de aumentar la cantidad de bits transmitidos por ciclo. Sin embargo, al elevar la velocidad aparecen problemas de sincronización entre líneas, interferencias y dificultad para mantener señales limpias.

Las interfaces serie modernas utilizan técnicas de señalización diferencial, codificación, enlaces punto a punto y tasas de símbolo muy elevadas. PCI Express, SATA y USB son ejemplos en los que la transmisión serie ofrece gran ancho de banda con menos conductores y mejor escalabilidad.

La afirmación de que el paralelo siempre transmite más información por unidad de tiempo solo es cierta si se comparan líneas con la misma velocidad individual y sin considerar limitaciones físicas. En la práctica, aumentar la frecuencia de un bus paralelo hace muy difícil que todos los bits lleguen exactamente a la vez. Este desfase, conocido como skew, obliga a reducir la velocidad o añadir márgenes.

Además, muchas líneas paralelas próximas generan diafonía, consumen más pines y complican placas y cables. Un enlace serie diferencial puede operar a una tasa muy superior por línea, usar menos conductores y recuperar el reloj a partir de la señal. Incluso puede combinar varias líneas serie independientes, como los lanes de PCI Express, para escalar el ancho de banda.

Por eso USB, SATA y PCI Express sustituyeron a muchas interfaces paralelas anteriores. La lección importante no es que "serie sea siempre más rápida", sino que a altas velocidades la señalización serie permite construir enlaces más simples, fiables y rápidos que un gran bus paralelo compartido.

## 1.9. Placa base y organización física del sistema

La placa base integra y conecta los principales componentes de un ordenador. Reconocer sus zonas y conectores ayuda a relacionar los conceptos abstractos de CPU, memoria, buses y entrada/salida con un equipo real.

La placa base interconecta los principales componentes del equipo. El zócalo de CPU proporciona conexión mecánica y eléctrica con el procesador. Los bancos DIMM alojan la memoria RAM. Las ranuras PCI Express permiten instalar tarjetas de expansión, y los conectores M.2 y SATA se utilizan para almacenamiento y otros dispositivos.

Los VRM (Voltage Regulator Modules) convierten y regulan tensiones para alimentar CPU y otros circuitos. El chipset o PCH, cuando existe como componente separado, gestiona parte de las interfaces de E/S. También se encuentran el chip de firmware UEFI, la batería que mantiene el reloj y determinados ajustes, conectores ATX/EPS de alimentación, cabeceras USB y de audio, conectores de ventilador y pines del panel frontal.

La disposición concreta cambia entre modelos y generaciones. Para identificar una placa real debe consultarse su manual, porque allí se indican posiciones, límites de potencia, compatibilidad de memoria, reparto de líneas PCIe y restricciones que pueden aparecer al usar ciertos puertos simultáneamente.

# 2. Representación interna de la información y lógica digital - Parte II

## 2.1. De la información humana a la representación binaria

La segunda parte conecta el hardware con la forma en que los computadores representan información. El concepto clave es que un mismo patrón de bits puede adquirir significados distintos según el código o formato con el que se interprete.

Esta segunda parte estudia cómo se representa y procesa la información en un sistema digital y conecta esas representaciones con las operaciones elementales del hardware. Los contenidos se aplican directamente al desarrollo de software porque permiten interpretar tamaños de datos, direcciones, máscaras, códigos hexadecimales y operaciones lógicas de bajo nivel.

El propósito es conectar el mundo físico del hardware con los datos y operaciones que utiliza el software. Todo programa termina representado mediante bits, y las operaciones elementales se implementan mediante circuitos lógicos. Comprender esta relación ayuda a interpretar direcciones, máscaras de red, códigos hexadecimales, tamaños de datos y funcionamiento básico del procesador.

El bloque se organiza en una progresión propia: primero se revisan los sistemas de numeración y el valor posicional; después se trabajan conversiones y aritmética binaria; finalmente se relacionan esos conocimientos con puertas lógicas y circuitos digitales.

Ambos bloques están estrechamente relacionados. Los sistemas de numeración proporcionan una forma humana de escribir patrones binarios; las puertas lógicas son los elementos que físicamente transforman esos patrones. Así se pasa de una representación abstracta de números a la implementación de operaciones en hardware.

El punto de partida para estudiar la representación interna es comprender que un ordenador no almacena directamente letras, fotografías o números tal como los percibimos. Todo se codifica mediante patrones de bits, y el significado depende del formato o del acuerdo utilizado para interpretarlos.

Por ejemplo, el mismo conjunto de bits podría representar un número entero, una instrucción, un carácter o parte de una imagen si se interpreta con reglas distintas. Por eso es fundamental diferenciar el valor físico almacenado -bits- de la interpretación lógica que realiza el software.

Las personas utilizan muchos sistemas de representación: lenguaje natural, números decimales, símbolos, colores, imágenes o sonidos. Para comunicarnos necesitamos conocer el código utilizado. Un texto solo transmite información si emisor y receptor comparten el significado de sus símbolos.

Los computadores digitales utilizan estados físicos distinguibles que se abstraen como 0 y 1. Esos bits pueden agruparse para formar bytes y estructuras mayores. Mediante códigos y formatos se representan números, caracteres, imágenes, audio, instrucciones y cualquier otra información procesable.

El hecho de usar binario no implica que internamente todo sea literalmente un "cero voltios" o "un voltio". Los circuitos trabajan con rangos de tensión y los interpretan como niveles lógicos. El binario es una abstracción robusta para representar esos estados.

La idea central puede resumirse así: el hardware almacena y transporta patrones binarios, mientras que el software asigna significado a esos patrones mediante formatos, códigos y estructuras de datos.

Un número entero puede codificarse en complemento a dos; un carácter puede representarse mediante Unicode; una imagen puede describirse como píxeles con componentes de color; y un fichero ejecutable contiene instrucciones y datos codificados. La clave es que el bit por sí mismo no tiene significado: el significado lo aporta el contexto de interpretación.

Para expresar los mismos valores pueden utilizarse diferentes sistemas de numeración. Los humanos trabajan normalmente en decimal, mientras que la electrónica digital se describe con naturalidad en binario. Octal y hexadecimal son formas compactas de escribir grupos de bits.

Convertir entre sistemas no cambia el valor representado, solo cambia su notación. Igual que "diez", 10 en decimal y 1010 en binario pueden referirse al mismo valor, las conversiones permiten pasar de una forma conveniente para el hardware a otra más legible para las personas.

## 2.2. Sistemas de numeración posicionales

Para trabajar con información binaria es útil dominar primero la idea general de base y valor posicional. El sistema decimal sirve como referencia para generalizar después el mismo razonamiento a otras bases.

La base de un sistema indica cuántos símbolos diferentes se utilizan antes de necesitar una nueva posición. Decimal utiliza diez símbolos, binario dos, octal ocho y hexadecimal dieciséis. En hexadecimal, los valores del diez al quince se representan habitualmente con A, B, C, D, E y F.

Estos sistemas son posicionales: el valor de un dígito depende de su posición. Cada posición tiene un peso que es una potencia de la base. En decimal, las posiciones valen ..., 1000, 100, 10, 1; en binario valen ..., 8, 4, 2, 1.

Un sistema no posicional no asigna el valor mediante potencias según la posición de cada símbolo. Los números romanos son un ejemplo habitual, aunque incluyen reglas propias de suma y resta. En informática se emplean sobre todo sistemas posicionales porque facilitan cálculo y conversión.

En base 10 cada posición representa una potencia de diez. Por ejemplo, 3726,58 puede descomponerse como 3 x 10^3 + 7 x 10^2 + 2 x 10^1 + 6 x 10^0 + 5 x 10^-1 + 8 x 10^-2. La misma idea se aplica a cualquier base.

Esta descomposición es importante porque se generaliza a cualquier base. Para convertir un número binario, octal o hexadecimal a decimal se multiplica cada dígito por la potencia de su base correspondiente y se suman los resultados.

La parte fraccionaria también puede representarse en otras bases. En esta unidad se trabajará principalmente con enteros para consolidar primero el valor posicional y los procedimientos de conversión. Algunas fracciones que son exactas en decimal no lo son en binario, lo que explica ciertos errores de redondeo en programación con coma flotante.

Cuando se cuentan números en decimal se recorren los dígitos 0 a 9. Al intentar avanzar después de 9 ya no existe otro símbolo de una cifra, por lo que esa posición vuelve a 0 y se incrementa la posición situada a la izquierda. Ese mecanismo es el acarreo.

El mismo principio se aplica a cualquier base. En base b, cada posición admite dígitos desde 0 hasta b-1. Cuando se supera el máximo, la posición vuelve a cero y se transporta una unidad a la posición siguiente. Comprender este patrón facilita contar directamente en binario, octal o hexadecimal.

Los ceros a la izquierda no modifican el valor de un entero: 009 y 9 representan el mismo valor decimal. Se utilizan a veces para mantener una anchura fija o visualizar grupos de dígitos.

## 2.3. Binario, octal y hexadecimal

Binario es la base natural de los circuitos digitales, mientras que octal y hexadecimal ofrecen representaciones más compactas de los mismos patrones de bits y facilitan su lectura por parte de las personas.

El sistema binario utiliza únicamente 0 y 1. Las posiciones tienen pesos 1, 2, 4, 8, 16, 32, etc. Por ejemplo, 101101 en base 2 representa 32 + 8 + 4 + 1 = 45 en decimal.

Cada posición se denomina bit. El bit situado más a la derecha es el menos significativo o LSB, porque representa 2^0. El situado más a la izquierda dentro de una palabra es el más significativo o MSB. Estos términos aparecen con frecuencia en arquitectura, redes y protocolos.

Con n bits sin signo pueden representarse 2^n combinaciones diferentes, desde 0 hasta 2^n - 1. Por ejemplo, con 8 bits existen 256 combinaciones, de 0 a 255.

En binario cada posición solo admite 0 y 1. Al sumar una unidad a 1 se obtiene 0 en esa posición y se acarrea 1 a la posición de la izquierda. Así, después de 1 viene 10; después de 11 viene 100; y después de 111 viene 1000.

Este mecanismo es exactamente el mismo que en decimal, pero el acarreo se produce con mayor frecuencia porque la base es 2. Practicar secuencias binarias ayuda a reconocer patrones: los bits de menor peso alternan más rápidamente que los de mayor peso.

En hardware, incrementar contadores binarios es una operación fundamental y puede implementarse mediante circuitos sumadores construidos con puertas lógicas.

Para practicar el acarreo, puede obtenerse una secuencia de números consecutivos partiendo, por ejemplo, de 10110110 en base 2. Basta con sumar una unidad repetidamente y observar cómo se propagan los acarreos cuando se encuentran varios bits 1 consecutivos.

Los diez valores siguientes son: 10110111, 10111000, 10111001, 10111010, 10111011, 10111100, 10111101, 10111110, 10111111 y 11000000. El último salto muestra cómo un acarreo puede avanzar por varias posiciones.

El sistema octal es posicional, de base 8, y utiliza los símbolos 0 a 7. Cada posición tiene un peso 8^n. Por ejemplo, 635 en base 8 equivale a 6 x 8^2 + 3 x 8^1 + 5 x 8^0 = 384 + 24 + 5 = 413 en decimal.

Octal tuvo especial importancia en sistemas cuyos tamaños de palabra se agrupaban fácilmente en bloques de tres bits. Todavía aparece en ámbitos como los permisos de archivos Unix, donde cada dígito octal representa tres bits de permisos.

Su relación con binario es directa porque 8 = 2^3. Cada dígito octal puede sustituirse por exactamente tres bits.

En base 8 se cuentan los dígitos 0, 1, 2, 3, 4, 5, 6 y 7. Después de 7 se produce acarreo y aparece 10 en octal. Del mismo modo, después de 17 viene 20, y después de 77 viene 100.

Es importante no leer 100 en octal como "cien decimal". Su valor es 1 x 8^2 = 64 en decimal. La notación de la base evita ambigüedad cuando se mezclan varios sistemas.

Los ceros iniciales solo sirven para visualizar el acarreo y no forman parte necesaria del número.

Hexadecimal utiliza base 16. Como hacen falta dieciséis símbolos, se usan 0 a 9 y las letras A a F para representar los valores 10 a 15. Cada posición tiene peso 16^n. Por ejemplo, 2B7 en base 16 vale 2 x 16^2 + 11 x 16 + 7 = 695 en decimal.

Hexadecimal es muy usado en informática porque un dígito hexadecimal representa exactamente cuatro bits. Esto permite escribir patrones binarios largos de forma compacta y legible. Direcciones de memoria, códigos de color, direcciones MAC, valores de depuración y numerosos formatos utilizan esta notación.

No debe confundirse la letra con un carácter arbitrario: A significa el valor decimal 10, B el 11 y así hasta F = 15.

En hexadecimal la secuencia es 0, 1, ..., 9, A, B, C, D, E, F. Después de F se produce acarreo y aparece 10 en base 16, que equivale a 16 decimal. Después de 1F viene 20, y después de FF viene 100.

Este patrón es útil para interpretar direcciones y volcados de memoria. Por ejemplo, aumentar una dirección 0x00FF en una unidad produce 0x0100. El prefijo 0x se utiliza habitualmente en lenguajes de programación para indicar un literal hexadecimal.

Un byte, al contener 8 bits, puede escribirse exactamente con dos dígitos hexadecimales: desde 00 hasta FF, equivalentes a 0 hasta 255 decimal.

El binario refleja directamente los bits, pero se vuelve difícil de leer cuando hay muchas posiciones. Octal y hexadecimal permiten agrupar bits sin hacer cálculos complejos: tres bits por dígito octal y cuatro bits por dígito hexadecimal. Por eso la conversión entre estas bases y binario es inmediata mediante agrupación.

Hexadecimal es especialmente útil porque dos dígitos representan un byte y resulta fácil reconocer patrones. Un valor binario de 32 bits puede escribirse con solo ocho dígitos hexadecimales. Esto reduce errores humanos al copiar o comparar información.

La compactación puede verse con un ejemplo nuevo: 1101 1010 0111 0011 en base 2 se escribe DA73 en base 16. Dieciséis bits quedan representados con cuatro dígitos hexadecimales sin cambiar el valor.

## 2.4. Conversión entre bases y aplicaciones prácticas

Cambiar de base no modifica el valor representado; únicamente cambia su escritura. Dominar varios métodos de conversión permite pasar con seguridad entre notaciones usadas en programación, redes y sistemas.

Las conversiones tienen aplicaciones reales. Las máscaras de red se entienden como patrones de bits que separan parte de red y parte de host. La notación CIDR expresa cuántos bits iniciales pertenecen al prefijo. Comprender binario ayuda a calcular rangos, direcciones de red y broadcast en IPv4.

Las direcciones MAC se escriben normalmente como seis bytes en hexadecimal, por ejemplo 02:7C:4A:91:2D:B0. IPv6 utiliza 128 bits y se representa mediante grupos hexadecimales, lo que resultaría poco práctico de manejar en binario para uso cotidiano.

Hexadecimal también aparece en programación web: los colores CSS pueden escribirse como #RRGGBB, donde cada par es un byte hexadecimal para rojo, verde y azul. Por tanto, estas bases no son un ejercicio puramente matemático: son herramientas de lectura de información técnica.

Para convertir entre bases se emplean tres métodos principales. Entre binario y octal o hexadecimal se aprovecha la correspondencia directa de grupos de 3 o 4 bits. Para convertir un entero decimal a otra base se aplican divisiones sucesivas entre la base y se leen los restos en orden inverso. Para pasar de una base cualquiera a decimal se usa la suma ponderada de cada dígito por una potencia de la base.

Para convertir entre octal y hexadecimal suele ser más sencillo pasar por binario: cada dígito octal se transforma en tres bits y después se reagrupan de cuatro en cuatro para hexadecimal, o viceversa. Así se evita una conversión decimal intermedia.

Cada método debe entenderse, no memorizarse de forma aislada. Todos derivan del valor posicional de los sistemas de numeración.

Como 8 = 2^3, cada grupo de tres bits corresponde a un dígito octal. Como 16 = 2^4, cada grupo de cuatro bits corresponde a un dígito hexadecimal. Para enteros se agrupa desde la derecha, comenzando por el bit menos significativo. Si el grupo de la izquierda queda incompleto, se añaden ceros a la izquierda.

Por ejemplo, 101 110 001 en base 2 corresponde a 5, 6 y 1 en octal, es decir, 561 en base 8. Del mismo modo, 1100 0101 corresponde a C5 en hexadecimal. Los ceros añadidos a la izquierda para completar un grupo no alteran el valor.

Las tablas de tres y cuatro bits conviene dominarlas hasta reconocer las equivalencias de forma casi inmediata. Esto agiliza ejercicios y resulta útil para interpretar bytes y máscaras.

La conversión inversa consiste en sustituir cada dígito por un grupo de longitud fija. En octal se utilizan tres bits por dígito; en hexadecimal, cuatro. Así, 356 en octal se convierte en 011 101 110 en binario, y A3C en hexadecimal se convierte en 1010 0011 1100.

Los ceros iniciales de un grupo deben conservarse durante la conversión para no perder posiciones. Después, si se trata de un entero y se desea una forma mínima, pueden eliminarse los ceros situados completamente a la izquierda.

Este método es exacto y no requiere operaciones aritméticas, solo conocer la tabla de equivalencias.

Como práctica de conversión directa, pueden transformarse 526 y 731 en base 8, y 2A9 y 5C4 en base 16, sustituyendo cada dígito por su grupo binario correspondiente. Los resultados mínimos en binario son, respectivamente, 101010110, 111011001, 1010101001 y 10111000100.

En sentido inverso, 1010111010 en base 2 puede agruparse como 001 010 111 010 para obtener 1272 en octal y como 0010 1011 1010 para obtener 2BA en hexadecimal. Del mismo modo, 111001011 en base 2 equivale a 713 en octal y 1CB en hexadecimal.

Estos ejemplos muestran que el procedimiento depende de la base de destino: se agrupa de tres en tres para octal y de cuatro en cuatro para hexadecimal, siempre desde el bit menos significativo hacia la izquierda.

Para convertir un entero decimal a base b se divide el número entre b. El resto es el dígito menos significativo del resultado. Se vuelve a dividir el cociente entre b y se repite hasta que el cociente sea cero. Finalmente se leen los restos desde el último obtenido hasta el primero.

El procedimiento funciona porque cada división separa la contribución de una potencia de la base. En binario los restos solo pueden ser 0 o 1; en octal van de 0 a 7; y en hexadecimal los restos 10 a 15 se escriben A a F.

Una formulación más precisa que "hasta no poder dividir más" es "hasta obtener cociente cero". Esto evita dudas y proporciona un criterio mecánico para finalizar.

Como ejemplo de divisiones sucesivas, 93 decimal se convierte en 1011101 en binario. La comprobación mediante potencias da 64 + 16 + 8 + 4 + 1 = 93.

La comprobación es una buena práctica. Después de una conversión decimal-binario, reconstruir el valor con potencias de 2 permite detectar errores de copia o de orden de los restos. El mismo método de divisiones serviría para base 8 o 16 cambiando el divisor.

Aplicando el mismo procedimiento, 173 decimal se convierte en 10101101 en binario, 255 en octal y AD en hexadecimal. El valor 46 decimal se representa como 101110 en binario, 56 en octal y 2E en hexadecimal.

Para 299 decimal, el resultado es 100101011 en binario, 453 en octal y 12B en hexadecimal. Puede comprobarse que 0x12B = 1 x 256 + 2 x 16 + 11 = 299 y que 453 en octal = 4 x 64 + 5 x 8 + 3 = 299.

En un examen conviene escribir claramente la base o usar prefijos reconocibles para evitar que un mismo conjunto de dígitos se interprete de forma incorrecta.

En un sistema posicional de base b, un número de n dígitos se interpreta como la suma de cada dígito multiplicado por una potencia de b. El dígito situado más a la derecha multiplica b^0; el siguiente, b^1; y así sucesivamente.

La fórmula general resume ese principio y permite convertir desde binario, octal, hexadecimal o cualquier otra base. En hexadecimal debe sustituirse A por 10, B por 11, ..., F por 15 antes de efectuar los productos.

Este método es también la explicación matemática del valor posicional. No es una regla independiente: expresa formalmente por qué una posición a la izquierda vale b veces más que la anterior.

Como ejemplos de conversión a decimal: 1011011 en base 2 = 64 + 16 + 8 + 2 + 1 = 91; 356 en base 8 = 3 x 64 + 5 x 8 + 6 = 238; y B7A en hexadecimal = 11 x 256 + 7 x 16 + 10 = 2938.

Una forma útil de comprobar una conversión es resolverla por un segundo procedimiento independiente cuando sea posible. Por ejemplo, un resultado obtenido mediante divisiones sucesivas puede verificarse reconstruyendo el valor con potencias de la base.

Comprobar el resultado con una segunda estrategia -por ejemplo conversión con agrupamientos o una calculadora usada después del ejercicio- ayuda a detectar este tipo de fallos.

Otros ejemplos resueltos son: 101011 en base 2 = 43 decimal; 1101101 en base 2 = 109; 642 en base 8 = 418; y 317 en base 8 = 207. En cada caso se suman las potencias correspondientes a las posiciones de los dígitos.

En hexadecimal, 7D3 = 7 x 256 + 13 x 16 + 3 = 2003 decimal, y 2A6 = 2 x 256 + 10 x 16 + 6 = 678 decimal. Es importante convertir correctamente A-F a sus valores decimales antes de multiplicar.

Estos ejercicios consolidan una destreza básica: reconocer que la cadena de símbolos no tiene un valor absoluto sin conocer su base.

## 2.5. Aritmética binaria

Las operaciones aritméticas en binario siguen las mismas ideas de acarreo y préstamo que en decimal, pero utilizando únicamente los dígitos 0 y 1. Estas reglas son la base de los circuitos aritméticos del procesador.

La suma binaria sigue reglas sencillas: 0+0=0, 0+1=1, 1+0=1 y 1+1 produce 0 con acarreo 1. Si además existe un acarreo de entrada, deben sumarse los tres bits. La resta utiliza préstamos de forma equivalente a decimal: cuando se necesita restar 1 de 0, se toma una unidad de la posición siguiente, que en binario equivale a 2 en la posición actual.

XOR es una puerta lógica: su salida vale 1 cuando sus dos entradas son diferentes. En un medio sumador, XOR produce el bit de suma de dos entradas y AND produce el acarreo. Para sumar dos bits más un acarreo de entrada se utiliza un sumador completo, que combina varias puertas.

Los computadores suelen implementar la resta mediante complemento a dos y circuitos sumadores, en lugar de construir un camino aritmético completamente separado. Esta técnica será más relevante cuando se estudie representación de enteros con signo.

La multiplicación binaria funciona como la multiplicación decimal, pero cada dígito del multiplicador solo puede ser 0 o 1. Multiplicar por 0 produce una fila de ceros y multiplicar por 1 reproduce el multiplicando. Cada fila parcial se desplaza según la posición del bit y después se suman las filas.

La puerta AND realiza exactamente el producto de dos bits: 1 AND 1 = 1 y todas las demás combinaciones producen 0. Por eso aparece como bloque elemental en circuitos multiplicadores, aunque un multiplicador completo necesita muchas puertas y sumadores interconectados.

Los desplazamientos binarios también tienen significado aritmético: desplazar un entero sin signo una posición a la izquierda equivale, si no hay desbordamiento, a multiplicar por 2. Esta relación es importante en arquitectura y optimización.

## 2.6. Puertas lógicas, tablas de verdad y circuitos digitales

La lógica digital muestra cómo los valores binarios dejan de ser una notación abstracta y se convierten en señales que pueden combinarse físicamente para realizar decisiones y operaciones.

El estudio de las puertas lógicas conecta la representación binaria con el hardware capaz de procesarla. Una puerta lógica implementa una función booleana elemental sobre una o más entradas binarias. Al combinar muchas puertas pueden construirse sumadores, comparadores, registros, multiplexores y finalmente unidades funcionales de un procesador.

El estudio de puertas no pretende diseñar un microprocesador completo, sino comprender cómo operaciones aparentemente complejas se descomponen en decisiones lógicas simples. Es el puente entre el 0/1 abstracto y los circuitos electrónicos.

Un circuito digital representa información mediante niveles lógicos. De forma abstracta se habla de 0 (LOW) y 1 (HIGH). Físicamente no se exige un único voltaje exacto para cada valor, sino rangos que el circuito reconoce como nivel bajo o alto, dejando márgenes de ruido para mejorar la fiabilidad.

Las puertas lógicas reciben uno o varios niveles de entrada y generan una salida según una función booleana. Al interconectarlas se forman circuitos combinacionales y secuenciales. Los primeros dependen de las entradas actuales; los segundos incorporan estado y pueden recordar información.

Las operaciones aritméticas estudiadas antes pueden implementarse con estas puertas. Un sumador binario no "entiende números" como una persona: propaga estados eléctricos a través de una red de funciones lógicas diseñadas para producir los bits correctos de suma y acarreo.

Una tabla de verdad enumera todas las combinaciones posibles de entradas y muestra la salida de una función lógica. Con dos entradas existen 2^2 = 4 combinaciones; con tres, 2^3 = 8. Esta representación permite comprobar de forma sistemática el comportamiento de una puerta o circuito.

NOT invierte una entrada. AND produce 1 solo cuando todas sus entradas son 1. OR produce 1 cuando al menos una entrada es 1. XOR produce 1 cuando las entradas son diferentes. NAND, NOR y XNOR son las negaciones de AND, OR y XOR respectivamente.

NAND y NOR tienen especial interés porque son funcionalmente completas: es posible construir cualquier función booleana utilizando únicamente puertas NAND o únicamente puertas NOR. Esto muestra que diferentes implementaciones físicas pueden realizar la misma función lógica.

Como primer ejemplo propio, considérese la función C = (NOT A) OR B. Para obtener la salida se invierte primero A y después se aplica OR entre NOT A y B.

Su tabla de verdad es: A=0, B=0 -> C=1; A=0, B=1 -> C=1; A=1, B=0 -> C=0; A=1, B=1 -> C=1. Este ejemplo permite practicar una negación antes de combinar señales.

Para resolver circuitos mayores conviene calcular primero la salida de cada puerta intermedia y avanzar de izquierda a derecha. Esa estrategia evita intentar deducir la expresión completa de una sola vez.

Para un circuito algo mayor, puede utilizarse F = (A AND B) OR ((NOT A) AND C). Conviene calcular dos señales intermedias: X = A AND B e Y = (NOT A) AND C; finalmente F = X OR Y.

La tabla de verdad de este circuito es: 000 -> 0, 001 -> 1, 010 -> 0, 011 -> 1, 100 -> 0, 101 -> 0, 110 -> 1 y 111 -> 1. Otra forma de interpretarlo es que, cuando A=1, la salida sigue a B; cuando A=0, sigue a C.

Este segundo ejemplo introduce una metodología general: nombrar resultados intermedios, calcular cada puerta en orden y comprobar después la expresión equivalente. El mismo procedimiento permite analizar circuitos más grandes sin depender de una figura concreta.

## 2.7. Uso de documentación y recursos técnicos

El aprendizaje de sistemas informáticos debe apoyarse en fuentes fiables y actualizadas. Los simuladores y recursos web son útiles, pero conviene aprender a contrastarlos con documentación técnica y académica.

La consulta de recursos externos debe utilizarse para verificar conceptos y ampliar información, no para sustituir el razonamiento técnico. Cuando se necesiten especificaciones concretas conviene acudir a la documentación oficial del fabricante o del organismo responsable del estándar.

Para conceptos fundamentales deben priorizarse manuales oficiales, especificaciones de fabricantes y documentación académica contrastada. Los simuladores de puertas lógicas son especialmente útiles para experimentar: permiten modificar entradas, observar salidas y comprobar tablas de verdad sin necesidad de montar circuitos físicos.

Los enlaces pueden cambiar o quedar obsoletos con el tiempo. Por ello el aprendizaje no debe depender de memorizar una URL concreta, sino de saber qué concepto se desea consultar y cómo verificarlo en una fuente fiable.

## 3. Fuentes técnicas generales de consulta y verificación

Las explicaciones, ejemplos numéricos y ejercicios de este documento se han redactado de forma autónoma. Para verificar detalles de estándares y tecnologías concretas se recomienda consultar las fuentes oficiales correspondientes.

UEFI Forum — especificaciones UEFI: https://uefi.org/specifications

NVM Express — especificaciones NVMe: https://nvmexpress.org/specifications/

PCI-SIG — especificaciones PCI Express: https://pcisig.com/specifications

Para componentes concretos (placa base, memoria, fuente, almacenamiento o periféricos), debe consultarse además la documentación oficial del fabricante del modelo utilizado.